Software Factory — Brief Board & Goal-Driven Orchestration

Design proposal · August 2026 · Kanban-as-folders for multi-brief operation, with `/goal` as the tenacity harness

> **One idea, three layers.** The **brief** defines what "done" means. **/orchestrate** is the only actor — it does the work and moves the brief between folders. **/goal** does no work: it just keeps restarting orchestrator turns until the board shows the brief has left the active column.

1 · The board: `briefs/` (committed to git)

Folder location is the single source of truth for status. The brief file is the Kanban card and it moves; machine state (`session/<brief-id>/`: phases, tasks, logs) never moves and stays gitignored. Each brief carries its own requirements, progress log, and blockers — readable at a glance without opening `session/`.

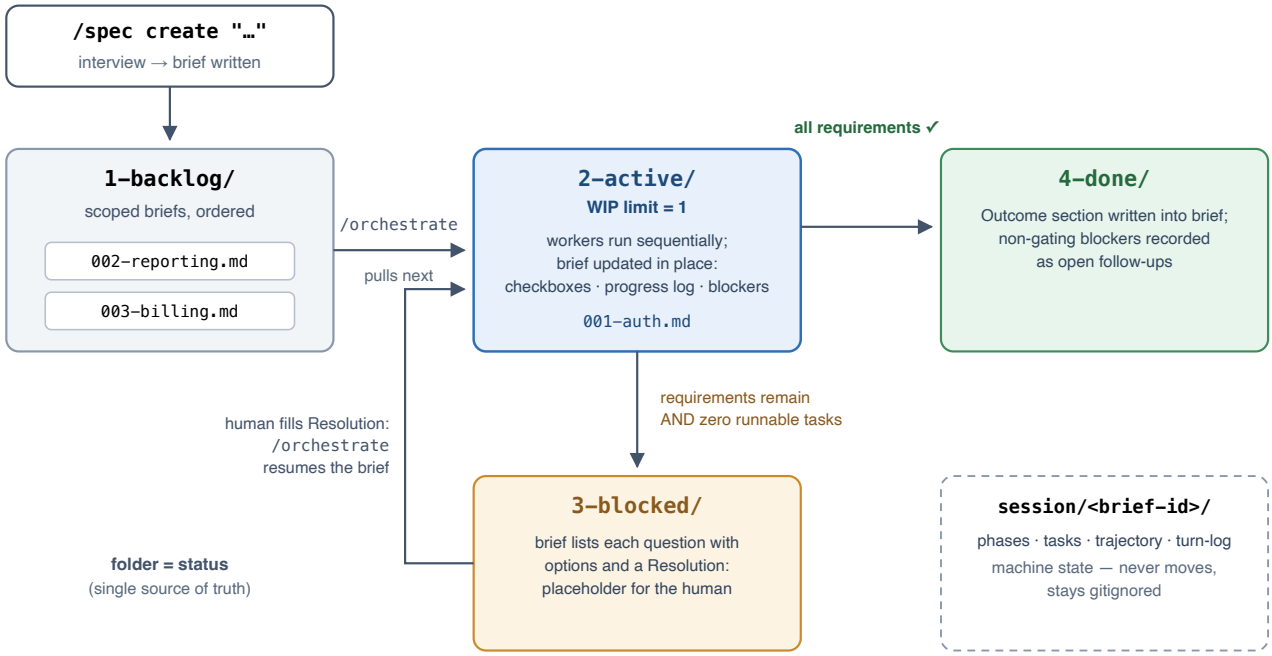


Figure 1 — Brief lifecycle across the board. Only `/orchestrate` moves the card.

2 · Routing rules

Route by **requirement state, not blocker existence**:

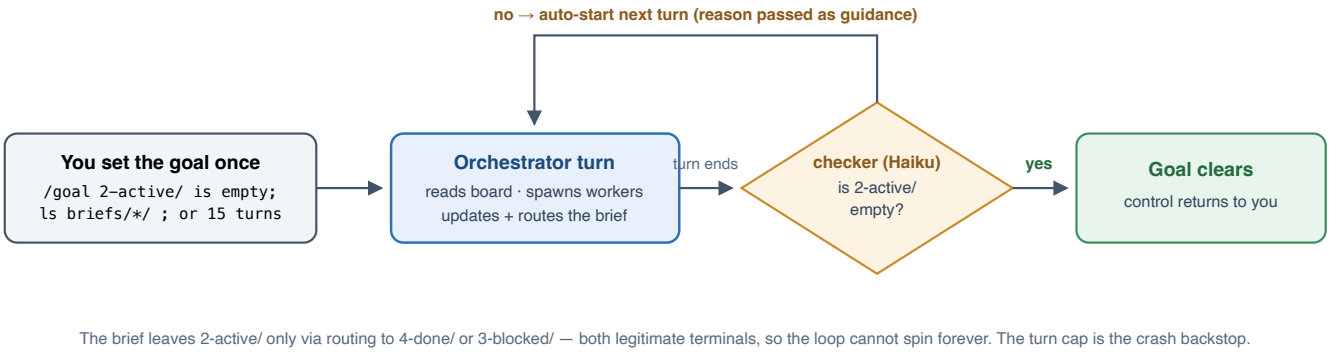
End-of-run state	Routes to
All requirements checked	`4-done/` — even if a non-gating blocker was logged; it is recorded in the brief as an open follow-up, not lost
Requirements remain + zero runnable tasks	`3-blocked/` — Blockers section lists each question with options
Human fills `Resolution:`, reruns `/orchestrate`	back to `2-active/` — blocked tasks re-planned with the answer

Park-and-continue: a human blocker does not stop the run. The orchestrator marks the affected task blocked and keeps executing every task not downstream of the blockage (by dependency, file ownership, or shared requirement). It routes to `3-blocked/` only when the runnable set is empty. The existing max-attempts escalation funnels repeated failures into the same path — so every run terminates in `done` or `blocked`.

Selection order for `/orchestrate`: `2-active/` (resume) → `3-blocked/` briefs with new resolutions → next `1-backlog/` brief.

3 · Where `/goal` fits: the ignition switch

`/goal` is a built-in Claude Code command, not an actor. After each turn a separate checker model (Haiku) reads the transcript and answers one structural question. "No" auto-starts the next orchestrator turn; "yes" clears the goal and returns control. The condition never needs the requirements in it — done-ness is delegated to the brief, and the board only changes when the orchestrator routes the card.



The brief leaves 2-active/ only via routing to 4-done/ or 3-blocked/ — both legitimate terminals, so the loop cannot spin forever. The turn cap is the crash backstop.

Figure 2 — The `/goal` ignition loop. Without it everything still works; you just retype `/orchestrate` after each turn.

4 · Why this cannot go infinite

- **"Blocked on human" is a success state of the goal**, not a failure the loop fights. Every non-terminating path (impossible task, repeated failure, unanswerable question) funnels into a blocker via max-attempts, which empties the runnable set, which routes the brief out of `2-active/` — satisfying the condition.
- **The condition is structural and transcript-provable**: the orchestrator prints `ls briefs/*/` every turn, so the checker judges one directory listing, not a claim.
- **Anti-gaming constraint** in the goal text: never weaken requirements or check a box without a completed task file.
- **Turn cap** (≈ 2 × phase count + 4) is the belt-and-suspenders backstop for crashes.

5 · Implementation touchpoints (factory changes)

File	Change
`onboard.py`	Scaffold `briefs/` + README; keep `session/` gitignored; add "Operating structure" section to `CLAUDE.md.tpl`
`factory/skills/spec/SKILL.md`	CREATE writes `briefs/1-backlog/NNN-slug.md` (drop one-spec-only error); SHOW renders the board; print the ready-made `/goal` line on approval
`factory/skills/orchestrate/SKILL.md`	Phase 0 → brief selection per routing order; paths → `session/<brief-id>/`; blocker handling → park-and-continue; new routing step moves the brief and writes Outcome; status proof (`ls briefs/*/`) at every turn checkpoint
`factory/skills/status/SKILL.md` + worker templates	Board rendering; path updates